Eric Romero
2/9/2025
CSC 445

# Homework 3 Writeup

**Description of the Algorithm:**

I will treat each input pair ($a_i$, $b_i$) as the line $y = a_i * x + b_i$. With this, compute the set of lines that form the upper envelope.

First, I bundle each line with its original index and sort the lines by increasing slope.

If multiple lines share the same slope, only the one with the largest intercept can be visible. I will discard all other duplicate lines except that line with the maximum intercept.

After this cleanup, the slopes will be considered strictly increasing. With this, I will build the upper envelope with a stack in one pass. Whilst processing the lines from smallest to largest slope, I will compute the intersection x-coordinates using: $x = (b_1 - b_2) / (a_2 - a_1)$. Each time I add a new line, I will compare the intersection point with the first visible point at the top line of the stack.

If the new line intersects the top line early, that top line can never be the maximum for any x, so pop it and repeat.

Once the intersection occurs strictly after the previous line's start point, the new line will take over later, so push it.

When the pass finishes, the stack contains the lines that are visible from above. The original indices are returned.

**Correctness:**

A line is visible from above exactly when it appears on the upper envelope, meaning it attains the maximum y-value for at least one real x.

First, If two lines have the same slope, the one with the larger intercept is always above the other for every x. So, the smaller-intercept line can never be maximal and not visible. Therefore, removing it is safe.

After sorting the remaining lines by increasing slope, the upper envelope can be built from left to right because envelope switching points occur at intersections and the visible lines must appear in increasing slope order.

The stack maintains candidate envelope lines in that order along with the x-value where each becomes topmost. When a new line intersects the current top line at an x that is less than or equal to the top line's start point, the top line has no interval where it can be the maximum:

It was not the topmost before its start point, and it is already beaten from its start point onward. So, popping it cannot remove a visible line.

Once the new line's intersection occurs after the previous start point, it creates a takeover later, so it is added with a valid start x.

At the end, every line left in the stack has a non-empty interval of x where it is the greatest among all lines, so every returned line is visible, and every non-returned line was removed only when shown to have no such interval. Thus, no visible line is missed.

Eric Romero
2/9/2025
CSC 445

**Complexity Analysis:**

**Let n be the number of input lines:**

**Time Complexity:**

1. Arranging & Sorting: List of n line objects (a, b, index) and sort by slope. Sorting will result in O(n logn)
2. Cleaning Up Duplicates: Single pass to keep maximum-intercept line per slope: O(n)
3. Stack Construction: Remaining lines are pushed onto the stack once, and possibly popped at most once. Payoff at the end: O(n)

Overall Runtime: O(n logn) + O(n) + O(n) = O(n logn)

**Space Complexity:**

1. Storing the lines is O(n)
2. Storing the stack of candidate lines: O(n)

Total Auxiliary space: O(n)